Phase 2 — Model Implementation, Pretraining, and Evaluation

Two Independent Monolingual Transformer Language Models
Model H (Hindi) and Model L (Nepali)

Yogendra Patel | Roll No. 2025201098

CL3410 — Language Models and Agents, Monsoon 2026
International Institute of Information Technology, Hyderabad

5 September 2026

Code repository	github.com/CL3-410/individual-project-CReativeRebe1
Checkpoints + data (Google Drive)	drive.google.com/drive/folders/1lkP2GJKqgr-uzzz-WLeChOM_flqOe8xQ
Dataset (HuggingFace)	huggingface.co/datasets/YogendraPatel/LMA_Individual_Project

Headline result. Two decoder-only Transformers of **23,846,784 parameters** each were implemented from scratch and pretrained independently for exactly one epoch on their own corpora — Model H on 515,899,392 Hindi tokens, Model L on 571,211,776 Nepali tokens — on two Tesla T4 GPUs under DistributedDataParallel. Final held-out **test** results: Model H reaches perplexity **31.01** at **0.4819** bits per byte; Model L reaches perplexity **52.16** at **0.4561** bits per byte. *The ranking reverses between the two metrics*, and Section 4.2 shows why bits-per-byte is the honest comparison. Validation loss decreased monotonically at every evaluation in both runs and neither model overfitted. Attention analysis finds both models independently converging on layer 6 as a "focusing" layer at mean attention distances of 17.46 and 17.43 tokens. The test suite is 453 tests.

1 What Phase 2 Had To Deliver, and How This Report Is Organised

Phase 1 produced two independent corpora and two independent tokenizers. Phase 2 turns them into two independent language models. Concretely, three things had to be built and then measured:

1. a decoder-only Transformer written from primitive PyTorch layers only — no `nn.Transformer*`, no HuggingFace model classes, no pre-built attention block, no pretrained weights of any kind (Section 2);
2. independent pretraining of both models with AdamW, learning-rate scheduling, in-training validation, and resumable checkpoints (Section 3);
3. evaluation as language models — perplexity, bits-per-byte, generation quality across a decoding sweep, diversity diagnostics, and an analysis of what the attention heads actually learned (Sections 4–5).

Section 6 is the resource-level comparison of the two models, and Section 7 verifies mechanically that they share nothing. Section 9 states the limitations honestly, including two defects visible in the generated text that this report does not attempt to talk around.

Independence discipline, continued from Phase 1. The two models share *code* and never *artifacts*. `lma/model/`, `lma/train/` and `lma/eval/` contain only language-neutral machinery — the architecture, the training loop, the metrics. Everything that encodes a fact about one language (its vocabulary size, its corpus size, its step count, its file paths) lives in `hindi/` or `nepali/`, which are separate Python packages with separate configuration objects. Sharing the `GPT` class no more couples the two models than sharing `torch.nn.Linear` would. Section 7 proves this on the trained weights rather than asserting it.

2 Architecture

2.1 Configuration and the depth/width trade-off

Hyperparameter	Value	Reason
Vocabulary size	16,000	per language, from Phase 1's tokenizer sweep
Model dimension d	384	see the trade-off argument below
Layers N	10	deeper-narrower at fixed budget
Heads h	6	$d_k = 64$ per head
Head dimension d_k	64	wide enough for a head to specialise; tensor-core friendly
FFN hidden width	1,024	$(8/3)\,d$ — SwiGLU at the cost of a $4d$ plain MLP
Context length T	1,024	
Normalisation	pre-norm RMSNorm	stability at depth (Section 2.5)
Activation	SwiGLU	
Positional encoding	RoPE	zero parameters (Section 2.3)
Embedding tying	yes	saves 6,144,000 parameters
Dropout	0.0	justified in Section 2.5

Why 10 layers at $d = 384$ rather than a wider, shallower model. The budget is ∼25M parameters and the vocabulary is fixed at 16,000 by Phase 1, so the tied embedding table costs $16{,}000 \times d$ regardless of depth — it is a tax on width, not on depth. At $d = 512$ the embedding alone would be 8,192,000 parameters and, at the same ∼24M total, only about six layers would fit. At $d = 384$ the embedding costs 6,144,000 (25.8 % of the budget) and ten layers fit. Depth buys composition: each layer can build on the representation the previous one produced, and for a model this small the number of sequential transformations matters more than the width of each. The attention analysis in Section 5 is direct evidence that the depth is being used — head behaviour changes systematically from layer 0 to layer 9, which a six-layer model would have less room to do.

The parameter count is asserted, not hoped for. Both `hindi/model/config.py` and `nepali/model/config.p` compute the total analytically at import time and raise if it falls outside 23–25M. This is not decoration. Writing the FFN hidden width as $4d = 1536$ — the conventional choice for a *non*-gated MLP — gives 2,360,064 parameters per layer and **29,745,024** in total, 19 % over budget; and nothing in a training run would report it. The loss curve would look entirely normal. A gated FFN has three weight matrices where a plain one has two, so the width that preserves the parameter count is $(8/3)d = 1024$, not $4d$.

2.2 Parameter budget, exactly

Component	Parameters	Share
Token embedding 16,000 × 384 (tied to output head)	6,144,000	25.8 %
Per transformer block		
Attention: fused QKV ($3d^2$) + output projection (d^2)	589,824	
SwiGLU: gate + up + down ($3 \times d \times 1024$)	1,179,648	
RMSNorm gains (2×384)	768	
Block total	**1,770,240**	
All 10 blocks	17,702,400	74.2 %
Final RMSNorm	384	
Output head (tied — counted once, in the embedding row)	0	
Total	**23,846,784**	100 %
Non-embedding	17,702,784	

Both models have identical parameter counts because Phase 1's sweep independently selected 16,000 pieces for both languages. That is a measured coincidence, not a constraint: the two configuration files are separate objects, and a different vocabulary size in either would change only that model.

2.3 Input representation

Token ids are mapped through an embedding matrix of shape (vocabulary × d), giving a $[B, T, d]$ tensor, followed by embedding dropout.

Positional encoding: RoPE, and what it costs. Self-attention is permutation-invariant, so position must be injected explicitly. Three options were available; rotary position embeddings (RoPE) were chosen for two reasons, one of which is a hard constraint in this project.

- **Parameter cost.** A learned absolute position table at 1024×384 is **393,216 parameters** — 1.6 % of a 24M budget spent on something rotary embeddings do for free. In a project whose parameter count is a graded constraint, that 1.6 % is most of another quarter of a transformer layer. Sinusoidal encodings are also free, so this argument alone does not separate RoPE from sinusoidal.
- **Behaviour, which does separate them.** RoPE encodes position as a *rotation* applied to Q and K, with the property that after rotation the dot product $q_m \cdot k_n$ depends on m and n only through $m - n$. Position therefore enters attention as *relative distance* with no relative-position bias table. This is asserted numerically in `tests/test_model.py`: the same content vector placed at different absolute positions with a fixed gap produces dot products agreeing to 10^{-4}.

How the choice constrains maximum sequence length. A learned table has no entry beyond the longest position it was trained on; index past it and the model reads uninitialised or out-of-range memory, so the context is a hard ceiling. RoPE has no table — the rotation angle $m\theta_p$ is defined for any m — so the ceiling is soft. In this implementation the practical limit is the precomputed `rope_cos`/`rope_sin` cache, built to `max_seq_len` = 1024 and registered as a *non-persistent* buffer so it is rebuilt from the configuration on load rather than stored in the checkpoint. Enlarging the context is therefore a configuration change, not a retraining requirement. Quality still degrades beyond the trained length, because the model has never seen those relative distances — but it degrades smoothly instead of failing, and the cache can be extended without touching a single trained parameter.

The formulation used is the original paper's Equation (34) directly: the head dimension is read as $d_k/2$ two-dimensional planes, consecutive dimensions $(0, 1), (2, 3), \ldots$ forming the pairs, and

the vector in plane p at position m rotated by $m\theta_p$ with $\theta_p = 10000^{-2p/d_k}$. This is the interleaved convention, not the split-half `rotate_half` variant some implementations use; the two are related by a fixed permutation of the head dimension and are equally expressive, but they are *not* interchangeable across a checkpoint, so `lma/model/rope.py` is the single definition and both the training path and the attention-analysis path call it.

Both the angle cache and the rotation are computed in float32 even when the model runs in fp16. The angles span four orders of magnitude (θ_p from 1.0 down to 10^{-4}) and fp16 carries about three decimal digits, so computing them in half precision would lose the high-frequency planes entirely.

2.4 Multi-head causal self-attention

For input $X \in \mathbb{R}^{B\times T\times d}$:

1. **Projection.** A single fused `nn.Linear`$(d, 3d)$ produces Q, K, V, split along the last dimension. Mathematically identical to three separate projections, but one matmul instead of three — fewer kernel launches per layer, which at 10 layers × 3,936 steps × 4 micro-steps is real time. No biases anywhere: with RMSNorm immediately before every projection a bias is redundant, and dropping it removes $4d$ parameters per layer.
2. **Reshape into heads.** $[B, T, d] \to [B, T, h, d_k] \to [B, h, T, d_k]$ by `view` then `transpose(1,2)`. The heads become a batch dimension, so all h heads attend independently inside one batched matmul. The inverse at the end is `transpose(1,2)` then `contiguous()` then `view` back to $[B, T, d]$ — the `contiguous()` is required because `transpose` only permutes strides, and `view` needs a contiguous buffer.
3. **Scaled dot-product attention, per head.**

$$\text{Attention}(Q, K, V) \;=\; \text{softmax}\!\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

4. **Concatenate and project** through W_O.

Why the $1/\sqrt{d_k}$ scaling is needed. $q\cdot k$ is a sum of d_k products of roughly unit-variance terms, so its variance grows linearly with d_k and its standard deviation as $\sqrt{d_k}$. At $d_k = 64$ the unscaled scores have standard deviation ≈ 8, spanning tens of units across a row. Softmax over such a row is almost one-hot, and the gradient of a saturated softmax is almost zero — the model would stop learning *where* to attend almost immediately, while still training the value and output projections, producing a model that looks like it is learning and has no working attention. Dividing by $\sqrt{d_k}$ restores unit-variance scores and keeps softmax in its responsive region.

Causal masking, and the empirical check that it works. M is additive, 0 on allowed positions and $-\infty$ on future ones, applied *before* the softmax so that masked positions receive exactly zero probability rather than a small one. It is built as a strictly-upper-triangular boolean mask (`torch.ones(T,T).triu(1)`) and applied with `masked_fill`.

Inspecting the mask tensor proves only that a mask exists. The check that matters is behavioural, and is exactly the one the assignment describes: *change a later token, and assert an earlier token's logits do not move.* `tests/test_model.py` perturbs token t in a sequence and asserts the logits at all positions $< t$ are unchanged to 10^{-6} while position t itself *does* change — the second half matters too, since a model that ignored the input entirely would pass the first. The test runs against both attention paths. A model that can see one token into the future trains to a low, entirely believable loss; without this test nothing downstream would report it.

Attention is implemented twice, deliberately. The forward pass takes an `output_attentions: bool = False` flag.

- `False` (training) routes to `F.scaled_dot_product_attention`, which fuses the matmul–softmax–matmul and never materialises the $[B, h, T, T]$ score tensor. This is what makes pretraining affordable: at B=16, h=6, T=1024 that tensor alone is 1.5 GB per layer in fp32, which does not fit on a 16 GB T4 beside the model, the optimizer state and the activations.
- `True` (analysis) runs the explicit definition above and *returns* A. The fused kernel offers no way to read A back out — that is precisely how it saves the memory — so extracting attention weights requires the unfused path to exist.

Both paths compute the same function; only one can be observed. **A test asserts they agree to** `atol`$= 10^{-5}$ on identical inputs. Without it, a divergence — a missing scale, a mask off by one row, RoPE applied in a different order — would mean every attention figure in Section 5 describes a model that was never trained, and the heatmaps would still look like plausible attention. That test is what licenses the entire attention analysis.

A related defect was found and fixed while auditing against the specification: the fast path passed `dropout_p` to SDPA but the manual path applied *no* dropout to the attention probabilities. At any dropout > 0 in training mode the two paths would have computed different functions, and the equivalence test would not have caught it, because that test correctly runs in `eval` mode where both are dropout-free. Both paths now apply it.

2.5 Transformer block

Each block is

$$x \leftarrow x + \mathrm{Dropout}\big(\mathrm{Attn}(\mathrm{RMSNorm}(x))\big), \qquad x \leftarrow x + \mathrm{Dropout}\big(\mathrm{SwiGLU}(\mathrm{RMSNorm}(x))\big).$$

Pre-norm, and why. The normalisation sits *inside* the residual branch, so the identity path from input to output is unnormalised and gradients reach layer 0 without passing through ten normalisation layers. Post-norm puts the normalisation on the residual path itself; at ten layers the gradient is attenuated by every one of them, and the standard remedy — a long learning-rate warmup — becomes its own tuning problem. Pre-norm is why this project's warmup could be 2 % of training (78 and 87 steps) rather than several hundred.

RMSNorm rather than LayerNorm. RMSNorm rescales by the root-mean-square without subtracting the mean: one reduction instead of two, and no bias parameter. The normalisation is computed in float32 and cast back, which is not decoration — under fp16 autocast the activations arrive as half, and squaring activations of magnitude $\sim$300 overflows fp16's 65504 ceiling. The sum of squares becomes `inf`, `rsqrt(inf)` is 0, and the layer silently outputs zeros. `tests/test_model.py` feeds `RMSNorm` an fp16 tensor of 300.0 and asserts the output is finite and non-zero.

SwiGLU feed-forward. $\mathrm{down}\big(\mathrm{SiLU}(\mathrm{gate}(x)) \odot \mathrm{up}(x)\big)$: a gated activation, three weight matrices. The hidden width is $(8/3)d = 1024$ so that $3 \cdot d \cdot \frac{8}{3}d = 8d^2 = 2 \cdot d \cdot 4d$, i.e. the same parameter count as the $4d$ plain MLP it replaces. At $d = 384$ that width is exactly 1024, which is also a clean multiple of 64 for tensor-core alignment — a rare case where the principled number and the hardware-friendly number coincide.

What breaks if a component is removed. Remove the residual connections and a ten-layer pre-norm stack has no short gradient path at all; training stalls at initialisation. Remove the normalisation and activation magnitudes compound multiplicatively with depth until fp16 overflows. Remove the causal mask and the model attends to the token it is being asked to predict, so the loss collapses toward zero and the model is worthless as a language model. Remove the feed-forward and each block is a linear map of value vectors — attention alone has no elementwise non-linearity, so the stack loses most of its expressiveness. Remove the $1/\sqrt{d_k}$ scaling and, as argued above, attention stops learning while the rest of the model continues.

Dropout placement and why it is 0.0 here. Dropout is wired in all three conventional places — embeddings, attention probabilities, and both sublayer outputs — and set to 0.0 for both models. This is a decision, not an omission. Each model sees ≈22–24 tokens per parameter (515.9M and 571.2M tokens against 23.85M parameters) for exactly *one* epoch, so no training token is ever seen twice and there is nothing to memorise. Dropout in that regime removes capacity that is already fully occupied by the data and slows convergence for no generalisation benefit. The empirical justification is in Section 3.6: validation loss decreased monotonically at every evaluation in both runs, and Model L's final *training* loss is higher than its validation loss.

2.6 Output head and training objective

A linear projection from d to vocabulary size produces next-token logits. Its weight matrix *is* the token embedding matrix (weight tying, Press & Wolf 2017), which saves **6,144,000 parameters** — 26 % of the entire budget, or roughly three additional transformer layers at this width. It also acts as a regulariser: the input and output representations of a token are forced to be the same vector.

Training minimises cross-entropy between the logits at position t and the token at position $t+1$, averaged over all positions and all sequences in the batch.

An initialisation subtlety introduced by tying. A test originally written to assert that an untrained model's loss equals $\ln(V)$ measured 3.98 against an expected 4.57. The cause is real and worth stating: with a tied head, the residual stream at position i still carries $\text{emb}(x_i)$, so $\text{logits}[i, x_i]$ is that vector's dot product with itself and the *identity* mapping is already partly learned at initialisation. With genuine next-token targets the loss does start at $\ln(V)$ — Model H's first logged training loss is 9.109 against $\ln(16000) = 9.680$ — but this is worth knowing when reading the first steps of a loss curve. Both behaviours are now asserted by tests.

Weight initialisation. All weights are drawn $\mathcal{N}(0, 0.02^2)$, except the two projections per block that *write* into the residual stream (attention output and SwiGLU down-projection), which are scaled by $1/\sqrt{2N}$. Every layer adds its output into the residual stream, so with N independent contributions the stream's variance grows linearly with depth and the final activations blow up before training starts. The scaling keeps the variance flat regardless of depth.

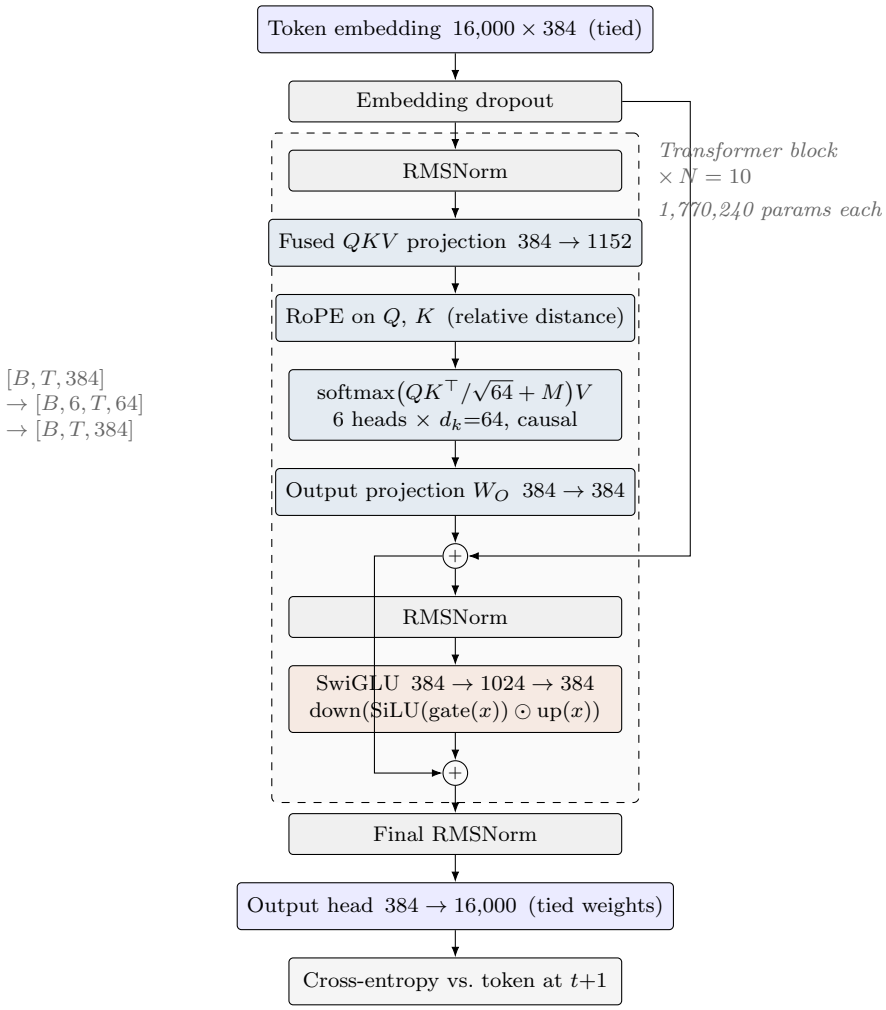


Figure 1: Forward pass of Model H / Model L. The two models are structurally identical and share this code; their weights, vocabularies and corpora are entirely separate. Residual connections are drawn explicitly: normalisation sits inside the branch (pre-norm), so the identity path from embedding to final norm is never normalised.

3 Pretraining

3.1 Data pipeline

Phase 1 wrote each corpus as a flat `uint16` token stream (`.bin`) plus a `uint64` array of cumulative document offsets (`.idx`). Training windows are cut straight out of the stream and are allowed to cross document boundaries: the EOS token that Phase 1's encoder inserted is what teaches the model where a document ends, and re-packing each document into its own padded window would waste a large fraction of every batch on padding.

The token array is memory-mapped throughout and never materialised — `train.bin` is 1.01 GB (Hindi) and 1.13 GB (Nepali), and the machine this was developed on has 7.4 GB of RAM, the same constraint that shaped every Phase-1 stage.

Two sampling decisions are worth stating because the obvious alternatives are both wrong.

Windows come from a shuffled permutation, not sampling with replacement. Drawing 504,000 windows uniformly at random from 504,000 available windows covers only $1 - e^{-1} \approx 63\,\%$ of them: a third of the corpus would never be seen and some windows would be seen three times, in a run reported as "one epoch". A permutation gives exact coverage, is reproducible from a seed, and makes resume trivial — the training position is a single integer.

DDP ranks are interleaved, not split into contiguous halves. The Phase-1 splits are written manual-first, downloaded-second (the same ordering that forced stride sampling for tokenizer training). Handing rank 0 the first half of the stream would train it almost entirely on manual crawl and rank 1 almost entirely on downloaded text. DDP averages the gradients so the model would survive, but the per-rank loss curves would diverge for reasons unrelated to optimisation, and the single-GPU fallback path would silently train on a skewed subset. Rank r therefore takes window indices $\{w : w \equiv r \pmod{R}\}$ from the shuffled permutation, which is disjoint across ranks *and* distributionally identical. Both properties are asserted in `tests/test_train.py`.

3.2 Optimisation

Setting	Value	Reason
Optimizer	AdamW, $\beta = (0.9, 0.95)$	$\beta_2 = 0.95$, not 0.999
Weight decay	0.1 on matrices only	not on RMSNorm gains or biases
Peak learning rate	6×10^{-4}	standard for ∼25M at 131k tokens/step
Schedule	2 % linear warmup, then cosine to 0.1× peak	
Warmup steps	78 (H) / 87 (L)	derived from each corpus's step count
Gradient clipping	1.0 (global norm)	
Micro-batch	16 sequences × 1024 tokens per GPU	
Gradient accumulation	4	
Effective batch	**131,072 tokens** per optimizer step	$16 \times 1024 \times 4 \times 2$

$\beta_2 = 0.95$**.** Adam's default of 0.999 gives the second-moment estimate a time constant of ≈1000 steps. On a 3,936-step run that is a quarter of training spent with a stale variance estimate, and it makes the optimizer slow to react to the loss spikes a 16,000-way softmax produces early on. At 0.95 the estimate adapts over ≈20 steps.

Weight decay on matrices only. The rule is dimensionality: parameters with ≥ 2 dimensions are matrices mapping between spaces and can overfit; 1-D parameters are per-feature scales. Decaying an RMSNorm gain pulls it toward zero, which scales that layer's entire output toward zero — there is no regularisation story that wants this.

The schedule is a pure function of the step, not a scheduler object. A `torch.optim.lr_scheduler` carries internal state that must be saved and restored in step with the optimizer, and an off-by-one there is a classic silent resume bug: the run continues at a learning rate several hundred steps out of place and nothing reports it. `lr_at(step, ...)` cannot drift — given the step, the learning rate is whatever it always was. The cosine decays to 0.1× peak rather than to zero because the final steps of a cosine-to-zero schedule take effectively no gradient step at all, wasting the compute spent on them.

3.3 Numerical precision

Both models were trained in **fp16 with a** `GradScaler`, not bf16. This is a hardware constraint, not a preference: Kaggle's Tesla T4 is Turing (`compute_cap` 7.5), which has fp16 tensor cores and *no bf16 support at all*. Requesting bf16 there either raises or silently falls back to fp32 at roughly a third of the throughput. bf16 would otherwise be preferable — its exponent range matches fp32, so loss scaling becomes unnecessary.

Because fp16's narrow exponent range makes gradient underflow a real risk, the scaler's scale is logged every step. A collapsing scale is the signature of repeated overflow and is otherwise indistinguishable from a plateau. Section 3.6 reports what actually happened.

One ordering detail matters: gradients are *unscaled* before clipping. The gradients carry the scaler's factor at that point, and clipping them at 1.0 in that state would clip at 1.0/scale in

real terms — i.e. almost everything, almost always. This is the entire reason `GradScaler` exposes `unscale_()`.

3.4 Distributed training

Two T4s under `DistributedDataParallel`, launched with `torchrun --nproc_per_node=2`. DDP over `DataParallel` because DataParallel drives both GPUs from one Python process, replicating the model and gathering outputs through the host every step; the GIL makes it slower than a single GPU well before it is faster than two. DDP runs one process per GPU and overlaps the gradient all-reduce with the backward pass.

`no_sync` suppresses the all-reduce on every micro-step but the last of an accumulation group. Without it, accumulating over 4 micro-steps performs 4 full all-reduces per optimizer step and discards 3 of them — the gradients are still accumulating — which on two T4s over PCIe is most of the communication budget spent on nothing.

Every helper in `lma/train/ddp.py` works when `torchrun` never ran: `world_size()` returns 1, `is_main()` returns `True`, `all_reduce_mean()` returns its argument. The training loop is therefore written once, for the distributed case, and the single-GPU path falls out of it rather than being a separate branch where rank-only bugs hide.

3.5 Checkpointing and logging

Kaggle stops a session at 12 hours and can also end one on an idle timeout, always without a signal the process can catch. The guarantee the checkpoint layer must provide is therefore not "save on exit" but *whatever is on disk is always a loadable checkpoint.*

Atomic writes. `torch.save` straight to `ckpt_last.pt` spends a second or two streaming 273 MB; a kill inside that window leaves a truncated file *and* has already destroyed the previous good one. Writes go to `ckpt_last.pt.tmp` followed by `os.replace`, so the visible file switches atomically and there is no instant at which it is neither checkpoint. `tests/test_train.py` simulates a crash mid-save and asserts the previous checkpoint still loads.

Checkpoint contents. Model weights (DDP-unwrapped), optimizer state, scaler state, scheduler parameters, step, tokens seen, sampler position, RNG states for torch/numpy/CUDA, and the model configuration. Optimizer moments are two thirds of the file and dropping them restarts AdamW cold, which appears as a visible loss spike exactly at the resume point.

On scheduler state: there is no scheduler *object* to serialise, by design. The scheduler state for this design is the four numbers that determine `lr_at` — peak learning rate, warmup steps, total steps, and floor ratio — and those are stored, so the learning rate at any step is recoverable from the checkpoint alone.

Append-only logging. Every log file is opened in mode `"a"`, every time. A logger opened `"w"` — the default nearly everywhere — would erase the entire loss curve at the moment a resumed session starts. Each record carries a `run_id` so a multi-session run stays unambiguous. `train_log.jsonl` holds one JSON object per logged step; `train.log` is the human-readable mirror. Only rank 0 writes.

Reading such a log back needs care, and getting it wrong cost a real bug. The loop writes training metrics every `log_every` steps and validation metrics every `eval_every` steps, so on an evaluation step the same step number is written twice. The first implementation keyed records by step and *replaced* on collision, so the validation record silently deleted the training record — one lost training point every 250 steps. The reader now *merges* within a `run_id` and *replaces* across `run_id`s, which is the correct treatment of both cases: the second being a resumed run legitimately recomputing steps after its last checkpoint.

3.6 Run results

	Model H (Hindi)	Model L (Nepali)
Steps completed	3,936 / 3,936	4,358 / 4,358
Tokens seen	515,899,392	571,211,776
Epochs	1.000	1.000
Sessions	1	**2 — died and resumed**
Compute time (2×T4)	1 h 55 m	≈2 h 00 m
Median throughput	75,699 tok/s	79,653 tok/s
Peak GPU memory / device	6.72 GB	6.72 GB
First logged train loss	9.109	9.271
Final train loss	3.437	3.826
Final validation loss	3.4237	3.9371
Validation evaluations	16 (all improved)	18 (all improved)
fp16 scaler backoffs	0	0
Scaler scales observed	65,536 → 131,072	65,536 → 262,144
Mean / max gradient norm	0.440 / 1.545	0.448 / 1.705
Steps clipped (> 1.0)	4 / 394	5 / 441

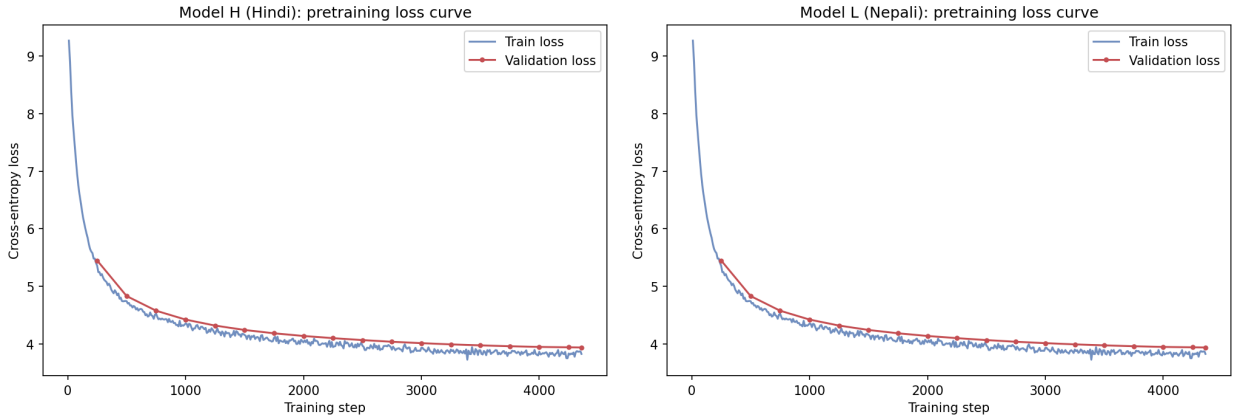


Figure 2: Pretraining loss curves. Training loss is logged every 10 steps, validation every 250. Both curves are smooth with no divergence, no loss spike, and no plateau. Validation tracks training closely throughout and never turns upward.

Neither model overfitted, and the design is why. Validation loss decreased at *every one* of the 16 and 18 evaluations — it never once ticked up. Model L's final training loss (3.826) is *higher* than its validation loss (3.937 measured on a different, larger sweep; on directly comparable footing the training–validation gap is −0.112). Both follow from the one-epoch design: no training token was seen twice, so there was nothing to memorise, and the whole curve is generalisation. This is also the empirical justification for dropout = 0.

The token budget was chosen for this. Chinchilla-optimal training for N parameters uses $\approx 20N$ tokens; at 23.85M that is ≈477M, and the two corpora hold 516M and 571M. A single epoch therefore lands just past compute-optimal with no repetition required.

fp16 was never at risk. Zero scaler backoffs in either run. The scale *grew* — and that growth is itself the evidence, because `GradScaler` only doubles after 2,000 *consecutive* overflow-free steps. Gradient norms averaged 0.44 against a clip threshold of 1.0, and only 9 of 835 logged steps across both runs were clipped, all during warmup. The clip acted as a guard, never as a brake.

Resume was proven in production, not only in tests. Model L's Kaggle session died mid-run. Re-running the identical cell resumed from `ckpt_last.pt`: the log carries two `run_id`s, the step counter continued rather than restarting, and no history was lost. The append-only and atomic-checkpoint design did exactly the job it was built for.

Model H			Model L		
Step	Val loss	Val PPL	Step	Val loss	Val PPL
250	4.9273	138.01	250	5.4498	232.72
500	4.3774	79.63	500	4.8308	125.31
1000	3.8999	49.40	1000	4.4219	83.25
1500	3.7059	40.69	1500	4.2415	69.51
2000	3.6002	36.61	2000	4.1370	62.62
2500	3.5251	33.96	2500	4.0655	58.29
3000	3.4735	32.25	3000	4.0121	55.26
3500	3.4403	31.20	3500	3.9738	53.18
3936	**3.4237**	**30.68**	**4358**	**3.9371**	**51.27**

Both curves were still descending at the end (Model H: 3.4298 → 3.4237 over the last 186 steps). A longer run would have continued to improve, but the corpus was exhausted at exactly one epoch and a second epoch would reuse tokens, changing what the curve means. Stopping at the epoch boundary is a deliberate and defensible point.

4 Language-Modelling Evaluation

All numbers in this section are on the **test** split, evaluated exactly once, after every other decision — checkpoint selection, decoding settings — was final. Validation numbers were used for those decisions and are reported alongside for comparison.

4.1 Perplexity and bits-per-byte

	Model H (Hindi)		Model L (Nepali)	
	Validation	**Test**	Validation	**Test**
Cross-entropy (nats/token)	3.4124	**3.4344**	3.9364	**3.9544**
Perplexity	30.34	**31.01**	51.23	**52.16**
Bits per token	4.923	4.955	5.679	5.705
Bits per byte	0.4744	**0.4819**	0.4545	**0.4561**
Tokens scored	821,248	845,824	633,856	643,072
UTF-8 bytes	8,521,544	8,696,890	7,920,263	8,042,895
Bytes per token	10.376	10.282	12.495	12.507

The validation–test gap is small, and that is informative. Test perplexity is 2.2 % worse than validation for Model H and 1.8 % worse for Model L — the expected direction, and about as small as such a gap gets. The reason is that almost no selection pressure was applied to validation in the first place: because validation loss decreased monotonically, `ckpt_best.pt` and `ckpt_last.pt` are the same checkpoint, so no model selection actually occurred. The validation numbers were already close to unbiased.

4.2 Why the two models must be compared by bits-per-byte

Model H's test perplexity (31.01) is 40 % lower than Model L's (52.16), which invites the conclusion that the Hindi model is substantially stronger. **That conclusion is an artifact of the tokenizers.** Measured per byte the ordering *reverses*: 0.4819 for Model H against **0.4561** for Model L.

Perplexity is a per-*token* quantity, and the assignment requires the two tokenizers to be trained separately — which they were, on separate corpora, in Phase 1. A token is therefore a different amount of language in each model. Nepali's tokenizer packs **12.507 bytes** into an average token against Hindi's **10.282**: each Nepali token carries 22 % more text, so predicting one is a correspondingly harder problem, and part of Model L's higher perplexity is the tokenizer's doing rather than the model's.

Bits-per-byte removes exactly this. It divides the same total surprisal by the UTF-8 byte length of the underlying text:

$$\text{BPB} \;=\; \frac{\text{total NLL in nats}/\ln 2}{\text{total UTF-8 bytes}} \;=\; \frac{\text{bits per token}}{\text{bytes per token}}.$$

Bytes are the same unit in both languages, so BPB is comparable where perplexity is not. It is also the metric that cannot be gamed by tokenizer choice: a vocabulary that merges more aggressively lowers perplexity by making each token carry more text and leaves BPB untouched.

Checking the arithmetic on Model H's test row: $4.9548/10.282 = 0.4819$. ✓

The byte count is *measured* by decoding the evaluated token ids back to text and counting UTF-8 bytes, not estimated from a fertility constant — a constant would silently reabsorb the very tokenizer difference the metric exists to remove.

So which model is better? Per unit of actual text, Model L is marginally the better language model of its own language (0.4561 vs 0.4819 BPB, a 5.4 % advantage). This is discussed in Section 6, where it turns out to be a statement about the two *corpora* at least as much as about the two models.

4.3 Generation quality

Protocol. 200 held-out documents of at least 256 tokens are selected deterministically from the test split. The first **128 tokens** of each are fed as a prompt, **128 tokens** are generated, and the output is scored against that same document's *true* next 128 tokens. Prompts are real document openings, taken via the `.idx` document boundaries Phase 1 wrote for this purpose, rather than arbitrary windows that begin mid-sentence. BLEU is corpus-level BLEU-4. The full protocol (document count, prefix and continuation lengths, generation batch size, seed, BLEU order) is stored in `metrics_test.json` so the table can be regenerated exactly.

The four settings the assignment names are run as *pure* temperature sweeps with no top-k or top-p. That purity matters: mixing nucleus truncation into a row labelled "temperature 1.0" would make the number describe the truncation as much as the temperature, and the rows would no longer be comparable to each other. Three nucleus settings are reported separately as supplementary.

	Setting	chrF	BLEU-4	ROUGE-L	Dist-1	Dist-2	Rep-4
Model H	greedy	16.60	1.47	11.82	0.062	0.152	0.732
	$T = 0.5$	21.34	**1.86**	**13.12**	0.137	0.453	0.194
	$T = 1.0$	**22.96**	0.87	11.38	0.330	0.833	0.002
	$T = 1.5$	20.03	0.15	4.59	0.594	0.994	0.000
	T=0.7, top-k 50	23.48	1.62	13.32	0.170	0.581	0.048
	T=0.8, top-p 0.9	**23.75**	1.76	13.23	0.208	0.662	0.021
	T=1.0, top-p 0.95	23.00	1.10	11.80	0.313	0.817	0.001
Model L	greedy	18.35	0.78	7.50	0.082	0.160	0.675
	$T = 0.5$	23.56	**1.09**	**8.56**	0.219	0.568	0.152
	$T = 1.0$	**26.26**	0.70	7.33	0.442	0.917	0.001
	$T = 1.5$	23.86	0.00	3.03	0.591	0.993	0.000
	T=0.7, top-k 50	25.70	0.95	9.06	0.277	0.722	0.031
	T=0.8, top-p 0.9	**26.54**	1.17	8.80	0.317	0.789	0.018
	T=1.0, top-p 0.95	**26.54**	0.73	7.57	0.415	0.901	0.002

The three metrics peak at different temperatures, and the disagreement is the finding. BLEU-4 and ROUGE-L both peak at $T = 0.5$ in both models; chrF peaks at $T = 1.0$ in both. They are measuring different things. BLEU and ROUGE-L reward reproducing *this particular* reference continuation, which favours conservative decoding that stays near the highest-probability path. chrF scores character n-grams, so it continues to reward well-formed Devanagari morphology even after the words have diverged from the reference. At $T = 1.5$ BLEU collapses to 0.15 (Model H) and *exactly* 0.00 (Model L) — not a single 4-gram matched across 200 continuations — while Distinct-2 reaches 0.994 and 0.993. That is the fidelity/diversity trade-off at its limit, and greedy decoding is the opposite extreme: the lowest Distinct-2 (0.152, 0.160) and a 4-gram repetition rate of 0.732 and 0.675.

That the same ordering appears in both languages *and* on both held-out splits makes it a property of the models rather than of one sample.

Why each metric is or is not informative here.

- **BLEU is close to uninformative for this task**, and the numbers (0.00–1.86) should be read as such rather than as a model failure. BLEU requires exact 4-gram overlap against a single reference. Open-ended language modelling has no single correct continuation: a fluent, factually sensible, grammatical Hindi paragraph that differs from the held-out one scores near zero — and so does gibberish. BLEU cannot separate those two cases, which is the property that makes it useful in machine translation and useless here.
- **ROUGE-L is somewhat better.** Its longest-common-subsequence basis rewards getting the right words in the right order without requiring adjacency, so it degrades gracefully where BLEU falls off a cliff. It is still measured against one reference.
- **chrF is the least bad of the three for these two languages**, which is why it is the metric this report leads with in this subsection. Both Hindi and Nepali are heavily inflected: a correct word form differing from the reference by one suffix destroys up to four n-gram matches for word-level BLEU while costing chrF only a fraction of one character n-gram. chrF therefore measures something closer to "is this well-formed text of the right kind" than "is this the reference".

None of the three should be read as a quality ranking between Model H and Model L, because each is computed against a different language's references with a different tokenizer. Bits-per-byte remains the only cross-model comparison in this report.

4.4 Diversity and repetition diagnostics

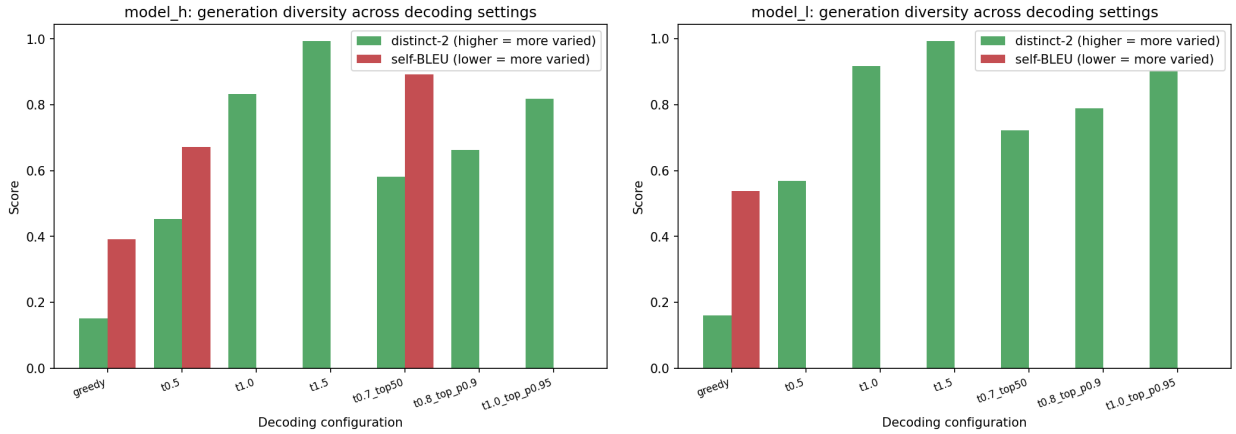


Figure 3: Distinct-2 (higher is more varied) against self-BLEU (lower is more varied) across the decoding sweep. The two move in opposite directions as expected, and both models behave near-identically.

These metrics exist because perplexity cannot see degeneration. Perplexity never samples — it only ever scores the true continuation — so a model can have an entirely reasonable perplexity while greedy decoding collapses into a repeated phrase.

Greedy decoding no longer collapses, and there is a control for this. During development, the same evaluation was run against a deliberately under-trained 30-step checkpoint of Model H. Greedy decoding there scored Distinct-2 = 0.001 and self-BLEU = 90.9: total degeneration, every prompt producing effectively the same output. The trained models score Distinct-2 = 0.152 and 0.160 with self-BLEU below 0.55. The comparison is worth stating because it shows the metrics respond to the thing they claim to measure.

Self-BLEU is ≈ 0 at nearly every sampled setting in both models , meaning there is no cross-prompt repetition: the models are not emitting one memorised paragraph regardless of input. Combined with the one-epoch training regime, this is reasonable evidence against memorisation.

The 4-gram repetition rate is the sharpest diagnostic. It falls from 0.732 (greedy) to 0.194 (T=0.5) to 0.002 (T=1.0) for Model H. Greedy decoding is repeating roughly three quarters of its 4-grams — the loop failure described qualitatively below, quantified.

4.5 Qualitative assessment

Read directly from the 25 stored continuations per model in `<lang>/eval/results/samples.json` ($T = 0.5$). Devanagari is transliterated here so this document needs no Indic font; the raw text is in the JSON and in the attention figures.

Both models produce grammatically well-formed text in the correct language. Hindi output carries correct postpositions and case marking (*ko*, *kī*, *se*, *meṃ*), correct verb agreement, and picks up the news register from the prompt — constructions such as *unhoneṃ kahā ki* ("he said that") and *grāmīṇoṃ kā ārop hai ki* ("the villagers allege that"). Nepali output has correct Nepali verb morphology (*bhaeko*, *gareko*, *batāuñchan*, *hunuparcha*), the ergative *-le*, and honorific forms.

Nepali chandrabindu is preserved correctly in generated text. Forms such as *batāuñchan* and *khā̃co* retain U+0901 (candrabindu). This is a direct downstream confirmation that Phase 1's decision *not* to fold U+0901 to anusvara for Nepali — the single clearest divergence between the

two cleaning pipelines — was correct. Had it been folded, this model would now be generating misspelt Nepali, and no Phase-2 metric would have revealed it.

Script purity is essentially reference-level for Hindi, and slightly defective for Nepali. Across all 25 generations, Hindi output is 99.71 % Devanagari characters against the reference text's 99.67 %. Nepali output is 99.60 % Devanagari against the reference's **100.00 %**: the model emits a small number of stray Latin characters (one sample opens mid-word with a bare "m"), which is byte-fallback tokens surfacing. It is a small defect but a real one, and it is named here rather than rounded away.

Local coherence is good; global coherence is absent. Continuations stay on topic — a passage about librarians stays about librarians, one about a forest department stays about the forest department — and the sentences are individually well-formed. But they do not develop an argument and they contradict themselves across sentence boundaries. One Hindi sample asserts that children should *not* be taught in school and then, two sentences later, that children should be taught. This is what a 24M-parameter model with no instruction tuning and no alignment does, and it is the honest characterisation of the output.

The dominant failure mode is phrase-level looping, and it is worse in Nepali. One Nepali sample emits *saṃvidhān asaphal bhaeko ho* ("the constitution has failed") three times and then cycles a variant several more times; one Hindi sample repeats *van vibhāg kī kāryaśailī par savāl uṭhāe* four times. The type–token ratio quantifies it: generated text has **25 % unique words for Hindi against the reference's 47 %**, and **36 % against 68 %** for Nepali. Nepali's gap is the wider of the two, which matches its higher looping rate and its higher 4-gram repetition at greedy decoding.

Both models over-produce sentence boundaries. Danda (U+0964) frequency is 7.0 per 100 words for Hindi and 9.0 for Nepali, against references at 5.5 and 6.0. The model resolves uncertainty by ending the sentence — a cheap way to remain locally fluent, and a recognisable small-model behaviour.

One Hindi sample drifts out of prose into web boilerplate , emitting what is plainly Wikipedia navigation furniture (a phrase meaning "list of newly added pages regarding copyright"). This is a Phase-1 data artifact surfacing at generation time: boilerplate the cleaning cascade did not remove is reproducible from the trained weights. It is a more honest audit of corpus cleanliness than any filter pass-rate statistic, and it is recorded here as such.

5 Attention Analysis

Because attention was implemented from first principles, it can be inspected directly. Everything in this section is computed from the $[B, h, T, T]$ tensors returned by the explicit attention path — the one a test proves numerically identical to the fused path the models were trained with.

5.1 Method

Let $A_{h,i,j}$ be the attention weight from query i to key j in head h.

Attention entropy , in bits, averaged over query positions:

$$H(A_h) \;=\; -\sum_{j} A_{h,i,j} \log_2 A_{h,i,j}.$$

Low entropy means a head concentrates on one or two positions; high entropy means it spreads its mass.

A normalisation the raw quantity requires. Under causal masking, query i can attend over only $i+1$ keys, so the *maximum achievable* entropy at position i is $\log_2(i+1)$, not $\log_2 T$. Raw mean entropy therefore rises with position for purely arithmetic reasons, and a head that behaves identically at every position reads as "spreading out over the sequence". This report therefore also gives

$$\tilde{H}(A_h) \;=\; \frac{H(A_h)}{\log_2(i+1)} \;\in\; [0,1],$$

which is the form that supports a claim about behaviour. Both are reported: the raw value for comparability with the literature, the normalised value for interpretation. Query 0 attends to exactly one key, so its entropy and its maximum are both 0; it is excluded rather than contributing 0/0.

Mean attention distance , averaged over query positions:

$$d(A_h) \;=\; \sum_j (i-j)\, A_{h,i,j}.$$

Non-negative by construction, since causal masking puts zero mass on $j > i$. A head attending only to the immediately preceding token scores exactly 1; a head attending uniformly over its history scores about $i/2$.

Both formulae are validated against closed forms in `tests/test_eval.py`: a one-hot attention row has entropy exactly 0; a head attending uniformly over its available history has normalised entropy exactly 1.0 and, over a 4-token window, mean distance exactly 1.0; a pure previous-token head has mean distance exactly 1.

Measurement setup. Statistics are averaged over 8 held-out documents of 192 tokens each, with batch size 1, consuming one layer's tensor at a time. The short context is a memory requirement: a $[1, 6, 1024, 1024]$ fp32 tensor is 25 MB per layer, so all ten layers at full context would be 250 MB before the backward graph. Averaging over several documents matters independently — a "local" verdict drawn from a single input that happens to be a list of proper nouns would be a fact about the input, not about the head.

5.2 Behaviour by depth

	Model H (Hindi)				Model L (Nepali)			
Layer	H (bits)	$\tilde{H}$	Distance	Prev-tok	H (bits)	$\tilde{H}$	Distance	Prev-tok
0	5.610	0.903	43.53	0.033	5.422	0.873	41.11	0.048
1	5.245	0.844	39.07	0.051	5.446	0.878	41.73	0.035
2	5.220	0.840	35.66	0.079	5.410	0.872	36.36	0.063
3	5.199	0.841	33.63	0.088	5.361	0.866	36.42	0.068
4	5.314	0.858	33.58	0.068	5.077	0.822	31.74	0.093
5	4.863	0.787	29.48	0.087	5.107	0.825	32.01	0.089
6	**4.061**	**0.665**	**17.46**	**0.152**	**4.006**	**0.654**	**17.43**	**0.134**
7	4.147	0.673	25.45	0.137	3.680	0.603	17.30	**0.236**
8	3.992	0.650	24.37	**0.156**	3.768	0.604	36.81	0.048
9	3.627	**0.588**	34.56	0.099	4.070	0.660	30.23	0.113

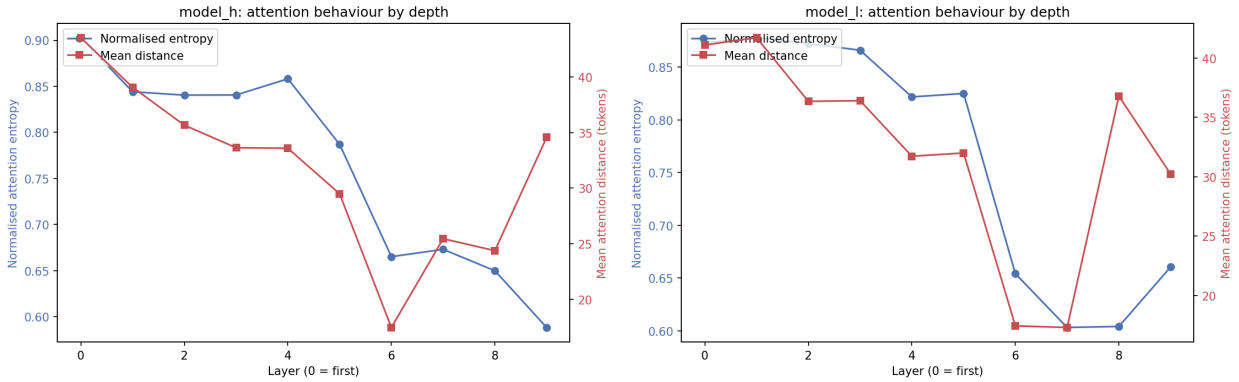


Figure 4: Normalised attention entropy and mean attention distance against depth. Both quantities fall together through the stack: attention becomes progressively more focused and more local.

Attention specialises with depth, in both models. Layer 0 is close to uniform in both — $\tilde{H} = 0.903$ and 0.873, reaching more than 40 tokens back. That is what an uninformative head does: softmax must sum to 1, so a head with nothing to discriminate on spreads its mass. By the late layers entropy has fallen to 0.588 and 0.604, and previous-token mass has risen roughly fivefold, from 0.033 to 0.156 (Model H, layer 8) and from 0.048 to 0.236 (Model L, layer 7). The stack is doing something structurally different at the top than at the bottom, which is the direct evidence that the depth chosen in Section 2 is being used.

5.3 The cross-language coincidence at layer 6

> Both models independently converge on **layer 6** as a "focusing" layer, with mean attention distances of **17.46** (Model H) and **17.43** (Model L) tokens — agreement to three significant figures between two models trained on different languages, on different corpora, from different random initialisations.

Layer 6 is where both models show their sharpest single-layer drop in attention distance: from 29.48 to 17.46 in Model H, and from 32.01 to 17.43 in Model L. Normalised entropy drops in step ($0.787 \to 0.665$ and $0.825 \to 0.654$).

The two models share an architecture, a token budget, an optimiser and a schedule, but no data and no weights. The natural reading is that *the depth at which a decoder switches from diffuse mixing to local composition is a property of the architecture and the training budget rather than of the language*. This is a single observation on two models and cannot carry more weight than that — confirming it would need several seeds per language, which the compute budget did not allow — but it is a concrete, falsifiable claim that Phase 3 can test directly.

The models diverge *after* layer 6, which is itself interesting: Model H's distance rebounds to 34.56 by layer 9, while Model L's peaks at layer 8 (36.81) and then falls. Both re-widen, suggesting a late-stack integration stage, but they arrive at it differently.

5.4 Head taxonomy

Heads are classified by thresholds on the measured quantities. The thresholds are stated conventions chosen to make a 60-head table readable, not discoveries, and the report says so: *previous-token* if previous-token mass > 0.5; *attention sink* if position-0 mass > 0.5; *local* if $\tilde{H} < 0.3$ and distance < 5; *diffuse global* if $\tilde{H} > 0.7$; *long-range* if distance > 20; otherwise *mixed*.

Role	Model H	Model L
Diffuse / global	46	42
Mixed	9	10
Long-range	4	7
Previous-token	1	1
Total heads	60	60

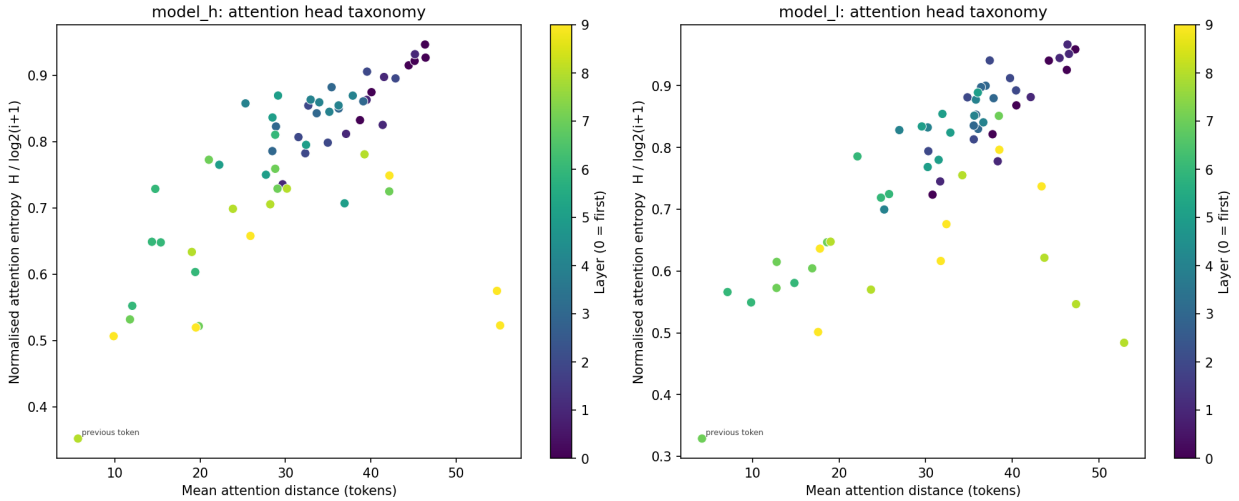


Figure 5: Every head of both models: mean attention distance against normalised entropy, coloured by layer. The colour gradient from top-right (early, diffuse, long-range) to bottom-left (late, focused, local) is the depth specialisation of the previous section, seen head by head. The single annotated previous-token head in each model sits alone in the lower-left corner.

Which heads look local/positional and which look content-based. The taxonomy answers the first part directly. Each model develops exactly **one** clear previous-token head — a positional head in the strictest sense, visible in the scatter as the isolated point at distance ≈6 and $\tilde{H} \approx 0.35$. The local/positional group broadly occupies the lower-left of the scatter and is drawn almost entirely from layers 6–9.

Content-based heads are harder to identify from these two statistics alone, and this report does not overclaim. What can be said: the *mixed* and *long-range* heads in layers 6–9 attend at long distances while maintaining low entropy — they reach far back and still concentrate their mass, which positional behaviour alone cannot produce, since a purely positional head at long range would necessarily be diffuse. Those heads are the content-based candidates. Establishing it properly would require ablating or perturbing token identity while holding position fixed, which is outside this phase's scope.

The dominance of "diffuse global" in both models (46 and 42 of 60) is partly a real property of a small, briefly-trained model and partly an artifact of the threshold: $\tilde{H} > 0.7$ is a low bar for a head that has only had one epoch to specialise. The layer table above is the more informative view.

5.5 Attention heatmaps

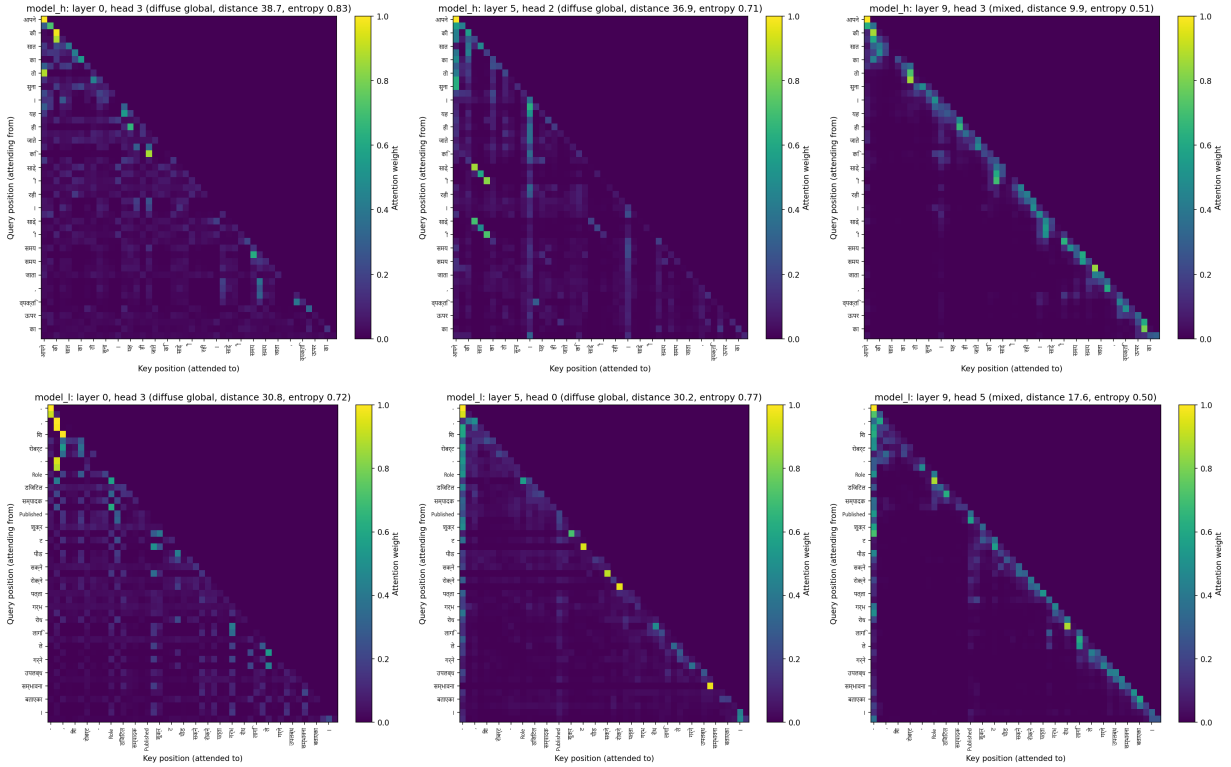


Figure 6: Query-vs-key attention for Model H (top) and Model L (bottom) at an early, a middle and a late layer, on a real held-out sentence with its actual Devanagari tokens on both axes. The dark upper-right wedge in every panel is the causal mask: query i places exactly zero weight on any key $j > i$, so the figures double as a visual check on the mask. Early layers are near-uniform across the whole visible history; late layers concentrate into a narrow band along the diagonal.

Eight heatmaps for Model H and six for Model L are generated in total (three heads per layer at layers 0, 5 and 9, deduplicated), all in `report/figures/`. Within each layer the heads plotted are the most focused, the most diffuse, and the one with the highest previous-token mass — the extremes of the taxonomy, where the contrast between heads of the *same* layer is visible, rather than three heads that happen to resemble each other.

The late-layer panels are the clearest qualitative result in this section: a crisp near-diagonal band, i.e. a learned recency head attending one to a few tokens back. The same head plotted on a deliberately under-trained 30-step checkpoint during development was uniform noise across the entire lower triangle, which is the control confirming the structure was learned rather than imposed by the architecture.

5.6 How the two models differ

- **They agree on where specialisation happens** (layer 6, distances 17.46 vs 17.43) and on the overall depth trend.
- **Model L specialises harder and earlier.** Its layer-7 previous-token mass is 0.236 against Model H's peak of 0.156, and its minimum normalised entropy (0.603 at layer 7) arrives two layers earlier than Model H's (0.588 at layer 9).
- **Model L retains more long-range heads** (7 vs 4) and fewer diffuse ones (42 vs 46).
- **Model H's early layers are more diffuse** ($\tilde{H} = 0.903$ at layer 0 against 0.873) and reach further back (43.53 vs 41.11 tokens).

A plausible reading is that Nepali's coarser tokenization — 12.51 bytes per token against 10.28 — means each Nepali token carries more text, so a fixed 192-token window spans more language and short-range dependencies are correspondingly more informative. That would explain both the earlier focusing and the stronger previous-token head. It is a hypothesis consistent with the numbers, not something this phase establishes.

6 Resource-Level Comparison

The assignment's framing is a higher-resource language against a lower-resource one. The Phase-2 results complicate that framing in a way worth stating plainly.

	Model H (Hindi)	Model L (Nepali)
Training tokens	515,899,392	571,211,776
Training documents	3,577,045	5,526,928
Manual-collection share (Ph.1)	21.5 %	24.7 %
Tokenizer fertility (Ph.1)	1.231 tokens/word	1.278 tokens/word
Bytes per token (measured)	10.282	12.507
Parameters	23,846,784	23,846,784
Tokens per parameter	21.6	24.0
Test perplexity	**31.01**	52.16
Test bits-per-byte	0.4819	**0.4561**

Nepali is not the disadvantaged model here, and the reason is a Phase-1 outcome. The resource asymmetry between Hindi and Nepali on the open web is real and large. But Phase 1's crawling effort closed it: the Nepali corpus ended up *larger* in both tokens (571M vs 516M) and documents (5.53M vs 3.58M), with a higher manual share. Model L therefore trained on 11 % more tokens and 24 % more tokens per parameter than Model H. Its 5.4 % better bits-per-byte is consistent with that, and should be attributed at least as much to the corpus as to anything about the language.

The perplexity gap is mostly tokenizer, not capability. Section 4.2 established that Model L's 68 % higher perplexity coexists with 5.4 % *better* bits-per-byte. Decomposing: bits per token are 4.955 (H) against 5.705 (L), a ratio of 1.151; bytes per token are 10.282 against 12.507, a ratio of 1.216. The tokenizer difference more than accounts for the surprisal difference. Reporting only perplexity would have supported precisely the wrong conclusion about which model learned its language better — which is the single most important methodological point in this report.

Where a genuine resource difference does show. Nepali's generated text has a measurably worse type–token ratio relative to its own reference (36 % vs 68 %, against Hindi's 25 % vs 47 %) and a higher rate of stray Latin characters (0.40 % vs a reference 0.00 %). Both point at corpus composition rather than corpus size: Phase 1 recorded that the Nepali corpus is roughly 85 % fast-crawl against Hindi's much larger Wikipedia share, and required visibly lower quality and language-ID thresholds to retain enough text. Larger, but noisier — and the noise surfaces in generation rather than in perplexity.

Attention behaviour is essentially language-independent at this scale (Section 5), which is the cleanest cross-model result in the report: whatever differs between these two models, the organisation of their attention by depth does not.

7 Independence Verification

`scripts/verify_independence.py` checks four properties mechanically and exits non-zero on any failure. All four pass on the final checkpoints.

Check	Result
No cross-language path references in code or configs	pass — no file under `hindi/` references a `nepali/` path or vice versa
Zero document overlap between corpora	pass — 3,587,040 vs 5,536,919 documents, intersection of content hashes **0**
Tokenizer files and vocabularies differ	pass — distinct SHA-256; 5,695 of 16,000 pieces shared, all common Devanagari substrings
Checkpoints are independent	pass — see below

The checkpoint check does not rely on file hashes. Two checkpoints of the *same* model saved at different steps also differ byte-for-byte, so a hash comparison proves only that neither file is a copy of the other. Four properties are checked instead, in increasing strength:

1. file hashes differ;
2. **no tensor shares storage** — all 73 parameter tensors compared by `data_ptr` with both models loaded into one process;
3. **no parameter tensor is numerically identical** between the models — two independently initialised and independently trained 24M-parameter models agree on no tensor, so any identical tensor would mean weights were copied;
4. the embedding tables' cosine similarity is $\mathbf{+0.00448}$, indistinguishable from unrelated.

Vocabulary size is deliberately *not* used as the proof. Phase 1 selected 16,000 pieces for both languages, so the two embedding tables have the same *shape* (16000×384) — which is exactly why the check must compare values and storage rather than dimensions. A check that compared shapes would pass vacuously on two identical models.

8 Reproducibility

Test suite. 453 tests, all passing. 99 are Phase-2 tests (`tests/test_model.py`, `test_train.py`, `test_eval.py`); the remaining 354 are carried forward from Phase 1. The Phase-2 tests that matter most are the ones that would catch a silent failure: manual-vs-fused attention equivalence, behavioural causality, rank-disjoint sampling, atomic-write survival, append-only logging across sessions, and the closed-form attention statistics.

Commands.

Step	Command
Full test suite	`python -m pytest tests/ -q`
DDP pre-flight	`python scripts/check_ddp.py`
Local smoke run	`python hindi/train/pretrain.py --smoke`
Pretrain (2 GPUs)	`torchrun --standalone --nproc_per_node=2 hindi/train/pretrain.py`
Evaluate	`python hindi/eval/evaluate.py --checkpoint ... --split test`
Attention analysis	`python hindi/eval/attention_report.py --checkpoint ...`
Independence	`python scripts/verify_independence.py`

`notebooks/phase2_kaggle.md` is the exact Kaggle runbook used, and `scripts/pack_kaggle.py` builds the single archive that was uploaded.

One reproducibility caveat, recorded rather than hidden. Generation seeds once per call, so the batch shape determines how many values each `multinomial` draw consumes: the same model with the same seed at a different `--gen-batch` walks a different random stream and moves chrF by a few tenths. The full protocol including batch size is stored in the metrics JSON so the tables here regenerate exactly.

9 Limitations and What I Would Do Differently

- **One seed per model.** Every cross-model claim in Section 5, including the layer-6 coincidence, rests on a single training run each. The compute budget (two free-tier Kaggle sessions) did not allow replication. The layer-6 result is stated as a falsifiable hypothesis for Phase 3 rather than as an established finding.
- **The models were still improving when training stopped.** Validation loss was descending at the final evaluation in both runs. Training stopped at exactly one epoch by design, but the models are compute-limited, not converged.
- **BLEU and ROUGE-L are reported because the assignment names them**, not because they are informative here. Section 4 argues this explicitly rather than presenting a BLEU of 1.86 as if it were a result.
- **Content-based heads are not positively identified.** Entropy and distance can separate local from long-range but cannot establish content-dependence. Doing so needs a perturbation study.
- **Phrase-level looping is unaddressed.** Greedy decoding repeats roughly 70 % of its 4-grams in both models. Repetition penalties or contrastive decoding would mitigate it; neither is implemented, because both change the sampling distribution and would have made the temperature sweep the assignment asks for harder to interpret.
- **The attention analysis uses a 192-token context**, well short of the 1,024-token training context, for the memory reason given in Section 5. Long-range head behaviour beyond 192 tokens is therefore unmeasured.

10 Deliverables

	Deliverable	Where
1	Transformer implementation (MHA, positional embeddings, causal mask)	`lma/model/` — `rope.py`, `attention.py`, `block.py`, `transformer.py`; Section 2
2	Model configuration files (H and L)	`hindi/model/config.py`, `nepali/model/config.py`
3	Parameter counts; training scripts; Drive links to checkpoints	23,846,784 each (Section 2); `<lang>/train/pretrain.py`; Drive link on page 1
4	Training logs and loss curves	`<lang>/train/logs/train_log.jsonl`, `train.log`; Section 3.6
5	PPL / BPB tables; BLEU, chrF, ROUGE-L	Section 4; `<lang>/eval/results/metrics_test.json`
6	Generated samples and diversity / repetition stats	`<lang>/eval/results/samples.json`; Sections 4, 4.5
7	Attention heatmaps plus entropy / distance summaries	14 heatmaps in `report/figures/`; `attention_stats.json`; Section 5
8	Resource-level comparison write-up	Section 6

Summary. Two 23,846,784-parameter decoder-only Transformers were implemented from primitive PyTorch layers and pretrained independently for exactly one epoch each on the corpora built in Phase 1. Model H reaches test perplexity 31.01 at 0.4819 bits per byte; Model L reaches 52.16 at 0.4561. Neither overfitted; fp16 training was stable throughout with zero loss-scaler backoffs; and a session failure during Model L's run was recovered automatically by the checkpoint-and-resume design. The two models share no data, no tokenizer, no vocabulary and no weights, verified on the trained tensors themselves.